

## Souvenirs

Амару хочет купить сувениры в местной лавке. Всего есть  $N$  **типов** сувениров. В лавке неограниченное количество сувениров каждого типа.

Сувенир каждого типа имеет фиксированную стоимость. Формально, сувенир типа  $i$  ( $0 \leq i < N$ ) стоит  $P[i]$  монет, где  $P[i]$  — целое положительное число.

Амару знает, что сувениры отсортированы в порядке убывания их стоимостей, более того все стоимости различны. Формально,  $P[0] > P[1] > \dots > P[N-1] > 0$ . Он также знает значение  $P[0]$ . Никакой другой информации о стоимости сувениров у Амару нет.

Для покупки сувениров, Амару будет производить транзакции определённого вида.

Каждая транзакция выполняется следующим образом:

1. Амару отдаёт некоторое положительное количество монет продавцу.
2. Продавец кладёт эти монеты стопкой на стол в задней комнате, где Амару не может их видеть.
3. Продавец рассматривает каждый тип сувениров  $0, 1, \dots, N-1$  в указанном порядке, один за другим. Каждый тип рассматривается **ровно один раз** в каждой транзакции.
  - При рассмотрении сувенира типа  $i$ , если текущее количество монет в куче составляет не менее  $P[i]$ , то
    - продавец забирает  $P[i]$  монет из стопки и
    - продавец кладёт на стол один сувенир типа  $i$ .
4. Продавец отдаёт Амару оставшиеся монеты и сувениры со стола.

Обратите внимание, что до начала транзакции на столе нет ни монет, ни сувениров.

Ваша задача — выполнить некоторое количество транзакций, чтобы:

- в каждой транзакции Амару купил **как минимум один** сувенир, и
- во всех транзакциях суммарно он купил **ровно  $i$**  сувениров типа  $i$ , для каждого  $i$  такого, что  $0 \leq i < N$ . Обратите внимание, что это означает, что Амару не следует покупать ни одного сувенира типа 0.

Амару не обязан минимизировать количество транзакций, у него есть неограниченный запас монет.

## Детали реализации

Вам следует реализовать следующую функцию:

```
void buy_souvenirs(int N, long long P0)
```

- $N$ : количество типов сувениров.
- $P0$ : значение  $P[0]$ .
- Эта функция вызывается ровно один раз для каждого тестового случая.

Вышеуказанная функция может вызывать следующую функцию, чтобы поручить Амару выполнить транзакцию:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- $M$ : количество монет, переданных Амару продавцу.
- Функция возвращает пару. Первый элемент пары — массив  $L$ , содержащий типы купленных сувениров в порядке возрастания. Второй элемент — целое число  $R$ , количество монет, возвращенных Амару после транзакции.
- Требуется, чтобы  $P[0] > M \geq P[N - 1]$ . Условие  $P[0] > M$  гарантирует, что Амару не купит ни одного сувенира типа 0, а  $M \geq P[N - 1]$  гарантирует, что Амару купит хотя бы один сувенир. Если эти условия не выполнены, вашему решению будет вынесен вердикт `Output isn't correct: Invalid argument`. Обратите внимание, что в отличие от  $P[0]$ , значение  $P[N - 1]$  не указывается во входных данных.
- Функция может быть вызвана не более 5000 раз в каждом тестовом случае.

Поведение грейдера **не адаптивно**. Это означает, что последовательность цен  $P$  фиксируется до вызова `buy_souvenirs`.

## Ограничения

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$  для каждого  $i$ , такого что  $0 \leq i < N$ .
- $P[i] > P[i + 1]$  для каждого  $i$ , такого что  $0 \leq i < N - 1$ .

## Подзадачи и баллы

| Подзадача | Балл | Дополнительные ограничения                                                                                                                                          |
|-----------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1         | 4    | $N = 2$                                                                                                                                                             |
| 2         | 3    | $P[i] = N - i$ для каждого $i$ , такого что $0 \leq i < N$ .                                                                                                        |
| 3         | 14   | $P[i] \leq P[i + 1] + 2$ для каждого $i$ , такого что $0 \leq i < N - 1$ .                                                                                          |
| 4         | 18   | $N = 3$                                                                                                                                                             |
| 5         | 28   | $P[i + 1] + P[i + 2] \leq P[i]$ для каждого $i$ , такого что $0 \leq i < N - 2$ .<br>$P[i] \leq 2 \cdot P[i + 1]$ для каждого $i$ , такого что $0 \leq i < N - 1$ . |
| 6         | 33   | Нет дополнительных ограничений.                                                                                                                                     |

## Пример

Рассмотрим следующий вызов.

```
buy_souvenirs(3, 4)
```

Существует  $N = 3$  видов сувениров и  $P[0] = 4$ . Обратите внимание, что существуют только три возможные последовательности стоимостей  $P$ :  $[4, 3, 2]$ ,  $[4, 3, 1]$ , и  $[4, 2, 1]$ .

Предположим, что `buy_souvenirs` вызывает `transaction(2)`. Предположим, что вызов возвращает  $([2], 1)$ , это означает, что Амару купил один сувенир типа 2 и продавец вернул ему 1 монету. Обратите внимание, что это позволяет нам сделать вывод, что  $P = [4, 3, 1]$ , поскольку:

- Для  $P = [4, 3, 2]$ , `transaction(2)` вернула бы  $([2], 0)$ .
- Для  $P = [4, 2, 1]$ , `transaction(2)` вернула бы  $([1], 0)$ .

Тогда `buy_souvenirs` может вызвать `transaction(3)`, который возвращает  $([1], 0)$ , это означает, что Амару купил один сувенир типа 1 и продавец вернул ему 0 монет. На данный момент, в общей сложности, он купил один сувенир типа 1 и один сувенир типа 2.

Наконец, `buy_souvenirs` может вызвать `transaction(1)`, который возвращает  $([2], 0)$ , это означает, что Амару купил один сувенир типа 2. Обратите внимание, что здесь мы могли бы также использовать `transaction(2)`. Таким образом, Амару в общей сложности купил один сувенир типа 1 и два сувенира типа 2, что и требовалось.

## Пример грейдера

Формат ввода:

$N$

$P[0] \ P[1] \ \dots \ P[N-1]$

Формат вывода:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Здесь  $Q[i]$  — количество сувениров типа  $i$ , купленных в общей сложности для каждого  $i$ , такого что  $0 \leq i < N$ .